Mechanics of Flexible Structures & Soft Robots
MAE-263F
Fall 2026

MS Comprehensive Exam: Extra Assignment

Implicit Euler vs. Newmark–Beta

Due Date: 11:59 PM, 12/11/2026

This assignment is only for students taking the UCLA MS Comprehensive Exam option. It must be completed **individually** (no teams).

Upload your work to your GitHub repository for this class, shared with the instructor (`khalidjm@seas.ucla.edu`). Do not create a separate repository. Create a new folder named `ms_comp` in your repository.

Submission Instructions

Your submission on BruinLearn should **only** contain the URL to the `ms_comp` folder of your GitHub repository, for example:

`https://github.com/<username>/<repository>/tree/main/ms_comp`

(Open the `ms_comp` folder on GitHub and copy the URL from your browser. If your default branch is not `main`, the URL will show its name instead.)

The `ms_comp` folder should contain exactly two files, with these names:

1. `NewmarkBetaAssignment.py`: your code (see *Code Requirements*).
2. `NewmarkBetaReport.pdf`: your report (see *Report*).

Make sure the instructor can open the link. If your repository is private, it must be shared with `khalidjm@seas.ucla.edu`; otherwise the assignment cannot be graded.

Problem

A ball of mass m hangs from a vertical linear spring of stiffness k under gravity g (Fig. 1). The ball may also feel a linear viscous drag force $-b\,\dot{y}$. Let y be the vertical position of the ball (positive upward), measured from the position where the spring is unstretched. The equation of motion is

$$m\ddot{y} = F(y, \dot{y}) \equiv -ky - b\dot{y} - mg, \tag{1}$$

and the ball is released from rest with the spring unstretched: $y(0) = 0$, $\dot{y}(0) = 0$. The total energy is

$$E = \frac{1}{2}m\dot{y}^2 + \frac{1}{2}ky^2 + mgy. \tag{2}$$

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Use the default values $m = 1\,\text{kg}$, $k = 10\,\text{N/m}$, $b = 0$, $g = 9.81\,\text{m/s}^2$, time step $\Delta t = 0.01\,\text{s}$, and total time $T = 20\,\text{s}$, unless stated otherwise.

Two exact solutions (for $b = 0$). These are used to check your code.

- **Free fall** ($k = 0$): $y(t) = -\frac{1}{2}gt^2$.
- **Undamped spring** ($k > 0$): $y(t) = y_{\text{eq}}\,(1 - \cos\omega t)$, with $\omega = \sqrt{k/m}$ and $y_{\text{eq}} = -mg/k$.

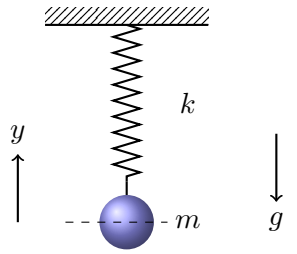


Figure 1: A ball of mass m on a vertical spring of stiffness k under gravity. y is positive upward; $y = 0$ when the spring is unstretched.

Time Integration

Implement both methods below. In both, solve each time step with the **Newton–Raphson method** (residual f and its derivative $J = \partial f/\partial y_{n+1}$), as in the codes from class, even though Eq. (1) is linear.

Implicit Euler. Find y_{n+1} such that

$$f = m\frac{y_{n+1} - y_n}{\Delta t^2} - m\frac{\dot{y}_n}{\Delta t} - F(y_{n+1}, \dot{y}_{n+1}) = 0, \qquad \dot{y}_{n+1} = \frac{y_{n+1} - y_n}{\Delta t}. \tag{3}$$

Newmark–Beta (Chapter 6 of the course notes). Find y_{n+1} such that

$$f = m\ddot{y}_{n+1} - F(y_{n+1}, \dot{y}_{n+1}) = 0, \tag{4}$$

where $\ddot{y}_{n+1}$ and $\dot{y}_{n+1}$ follow from the Newmark relations

$$y_{n+1} = y_n + \Delta t\,\dot{y}_n + \Delta t^2\left[\left(\tfrac{1}{2} - \beta\right)\ddot{y}_n + \beta\,\ddot{y}_{n+1}\right], \tag{5}$$

$$\dot{y}_{n+1} = \dot{y}_n + \Delta t\left[(1 - \gamma)\,\ddot{y}_n + \gamma\,\ddot{y}_{n+1}\right]. \tag{6}$$

The initial acceleration is $\ddot{y}_0 = F(y_0, \dot{y}_0)/m$. This assignment uses two choices of the parameters:

- **Newmark (0.25, 0.5):** $\beta = 0.25$, $\gamma = 0.5$ (average acceleration). These are the default values in the code.
- **Newmark (0.5, 1.0):** $\beta = 0.5$, $\gamma = 1.0$ (fully implicit Newmark). Run it with `--beta 0.5 --gamma 1.0`.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Code Requirements

1. Submit **one** Python file, `NewmarkBetaAssignment.py`. Start from the template provided with this assignment, `NewmarkBetaAssignment_template.py`. It contains two functions:

   - `main()`: reads the command-line options, calls `simulate()`, prints the results, and saves a figure. **Do not change it.**
   - `simulate(method, m, k, b, g, dt, T, beta, gamma)`: **you write this function.** It simulates the ball from $y(0) = 0$, $\dot{y}(0) = 0$ with `method` equal to `'euler'` (implicit Euler) or `'newmark'` (Newmark–Beta with `beta` and `gamma`; these two are ignored for implicit Euler), solving each time step with the Newton–Raphson method. It returns four `numpy` arrays, `t, y, v, E`, each of length $n+1$ with $n = \text{round}(T/\Delta t)$: the times $t_i = i\,\Delta t$, the position y, the velocity $\dot{y}$, and the total energy E of Eq. (2).

   You may add any other functions to the file and organize your code as you like.

2. Use only `numpy`, `matplotlib`, and the Python standard library. The file must not import any other file. It must not open a window (do not call `plt.show()`); figures are saved to files.

3. The program is run from a terminal as `python NewmarkBetaAssignment.py` followed by any of these options:

Option	Meaning	Default
`--method`	`euler` (implicit Euler) or `newmark` (Newmark–Beta)	`newmark`
`--m`	mass m [kg]	1
`--k`	spring stiffness k [N/m]	10
`--b`	drag coefficient b [N·s/m]	0
`--g`	gravity g [m/s^2]	9.81
`--dt`	time step Δt [s]	0.01
`--T`	total time T [s]	20
`--beta`, `--gamma`	Newmark parameters β, γ	0.25, 0.5

4. Each run prints one line with the final position, final velocity, and the initial and final energies, e.g.,

   ```
   method=euler m=1 k=10 b=0 dt=0.01 T=20 y_final=...  v_final=...  E_initial=...  E_final=...
   ```

   and saves a figure with two plots, y vs. t (with the exact solution when $b = 0$) and E vs. t. `main()` already does this.

5. The instructor will run, for example:

   ```
   python NewmarkBetaAssignment.py --method euler --k 0
   python NewmarkBetaAssignment.py --method newmark --k 0
   python NewmarkBetaAssignment.py --method euler
   python NewmarkBetaAssignment.py --method newmark
   python NewmarkBetaAssignment.py --method newmark --b 0.5
   python NewmarkBetaAssignment.py --method newmark --beta 0.5 --gamma 1.0
   ```

   Each command should finish in a few seconds.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Report

Submit a report, `NewmarkBetaReport.pdf`, of **at most 5 pages**, with your name on the first page.

Methods. Compare three methods: **implicit Euler**, **Newmark (0.25, 0.5)** ($\beta = 0.25$, $\gamma = 0.5$), and **Newmark (0.5, 1.0)** ($\beta = 0.5$, $\gamma = 1.0$). All runs use $b = 0$.

Figures. Every plot must have an x-label and a y-label, with units where applicable (e.g., t [s], y [m], E [J]), and a legend when it shows more than one curve. Number every figure, and discuss every figure in the text (e.g., "Figure 2 shows ...").

The report should have the following five sections.

1. **Model and Algorithm.** Write the equation of motion and the total energy, and explain why the energy is constant when $b = 0$. Write the residual f and its derivative $J = \partial f / \partial y_{n+1}$ for implicit Euler and for Newmark–Beta (with general β and γ), and give pseudocode for one time step of each.

2. **Verification with free fall** ($k = 0$). Plot y vs. t for the three methods together with the exact solution $y = -\frac{1}{2}gt^2$, and report $y(T)$ from each method and from the exact solution. Explain why both Newmark choices match the exact solution (up to round-off) while implicit Euler does not.

3. **Undamped spring** ($k = 10\,\mathrm{N/m}$). Plot y vs. t (with the exact solution) and E vs. t for the three methods. Explain the differences between them.

4. **Time step** ($k = 10\,\mathrm{N/m}$). Run the three methods with $\Delta t = 0.04$, 0.02, and $0.01\,\mathrm{s}$, and give a table of the final energy $E(T)$ (nine values). How does the energy of each method depend on Δt?

5. **Conclusion.** Summarize your findings. When would you use each method in a simulation of a flexible structure (e.g., the cantilever from class)?

Generative AI

As stated in the syllabus, you may use large language models (LLMs) and other generative AI tools to help write code. However, you are responsible for every line of code you submit and must be able to explain it when asked in person.

Academic integrity: this is an individual assignment. You are responsible for everything in your code and report. Including code, equations, results, or other content that you cannot explain when asked in person will be treated as a violation of academic integrity.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor